

SELENIUM PYTHON FRAMEWORK DEVELOPMENT

Project Report — Capstone Assignment 2

Web Application Automation Testing using Selenium WebDriver, Python, PyTest, Unittest, POM

Application Under Test: TutorialsNinja Demo E-Commerce Application

Programming Language: Python 3.12.3

Automation Tool: Selenium WebDriver 4.35.0

Testing Framework: PyTest 8.4.2

Reporting: pytest-html 4.1.1

Capstone Project Report

Academic Project Report

NAME: PRATEEK SINHA

ENROLLMENT NUMBER: 12023052020007

STREAM: CSE(AI)

INSTITUTE OF ENGINEERING AND MANAGEMENT, SALT LAKE

CERTIFICATE

This is to certify that Prateek Sinha, a student of Bachelor of Technology (B.Tech) CSE(AI), has successfully completed the academic project entitled “Selenium Python Framework Development” as part of the academic requirements of the B.Tech in Wipro Placement program. The project has been carried out under the guidance and supervision of ANINDITA DAS BHATTACHARJEE, Professor & Faculty Mentor, Department of CSE. The work presented in this report demonstrates the student’s practical understanding and application of Python programming, Selenium WebDriver, PyTest, Page Object Model (POM), data-driven testing, configuration management, automated test execution, error handling, logging, screenshot capture, and HTML test reporting. The project reflects the student’s efforts in applying software testing concepts and automation techniques to develop a structured, reusable, and maintainable testing framework. This report is submitted as a record of the academic work completed by the student and represents the application of theoretical knowledge to a practical software testing project under the guidance and supervision of the faculty mentor during the Academic Year 2026.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members and mentors for providing the guidance and support required to complete this project on Selenium Python Framework Development. Their technical guidance helped me understand the principles of web automation, test framework design, Page Object Model, PyTest, and software testing practices.

I am also thankful to the developers and maintainers of Selenium, Python, PyTest, and the other open-source technologies used in this project. Their documentation and tools provided the technical foundation required to develop and execute the automation framework.

I would also like to acknowledge the importance of practical experimentation during the development of this project. Working with a real browser automation environment provided valuable experience in identifying web elements, implementing explicit waits, debugging locator failures, managing test data, generating reports, and organizing automation code into reusable components.

Finally, I am grateful to everyone who contributed directly or indirectly to the completion of this project. The knowledge gained through this work has strengthened my understanding of software testing and provided practical exposure to automation framework development.

TABLE OF CONTENTS

1. Introduction
2. Project Objectives
3. Problem Statement and Business Scenario
4. Technologies and Tools Used
5. System Architecture
6. Project Structure
7. Page Object Model Implementation
8. Test Case Design and Automation Flow
9. Data-Driven Testing and Configuration Management
10. Error Handling, Logging and Screenshots
11. Test Execution and Results
12. HTML Reporting and Project Output
13. Advantages, Limitations and Future Scope
14. Conclusion
15. References

1. INTRODUCTION

Software applications have become an essential part of modern business operations, and ensuring their reliability, functionality, and usability is an important part of software development. Testing is therefore an integral stage of the Software Development Life Cycle (SDLC). Web applications, particularly E-Commerce applications, contain numerous functionalities that must be tested repeatedly whenever changes are introduced. Performing all these repetitive activities manually requires considerable time and effort. Automation testing provides an efficient solution by allowing frequently executed test scenarios to be performed automatically with consistent results.

The present project focuses on the development of a Selenium-based web automation framework using Python and PyTest. The framework is designed to automate important functionalities of an E-Commerce web application while following a structured and maintainable automation architecture. The TutorialsNinja Demo E-Commerce application is used as the application under test because it provides a suitable environment for demonstrating common E-Commerce operations such as user login, product search, product selection, and shopping-cart management.

The main purpose of the project is not simply to automate individual browser actions but to design a structured and reusable automation framework. A conventional Selenium script generally contains browser commands, locators, test data, assertions, and test logic in a single file. Such an approach becomes difficult to maintain as the number of test cases increases. In contrast, this project separates different responsibilities into dedicated components. Page-specific locators and actions are maintained in Page Object classes, test cases are stored separately, configuration information is maintained independently, and reusable utilities are provided for common framework operations.

The framework also incorporates PyTest fixtures, explicit waits, CSV-based test data management, configuration management, logging, failure screenshot capture, and HTML execution reports. These components make the framework more suitable for real-world automation practices. PyTest provides a convenient mechanism for organizing and executing test cases, while Selenium WebDriver provides browser automation capabilities.

The automation process begins by launching the Chrome browser and opening the selected E-Commerce application. The framework then executes the required test scenario through the corresponding Page Object classes. Login functionality is tested using both valid and invalid credentials. Product-related tests perform product searching, product selection, adding products to

the shopping cart, and updating product quantities. Assertions are used to verify whether the actual application behaviour matches the expected behaviour.

Another important objective of this project is to develop an automation solution that can be extended in the future. Additional test cases, browsers, pages, utilities, and CI/CD capabilities can be added without significantly changing the existing framework architecture. The framework can therefore serve as a foundation for more advanced automation projects involving Jenkins, GitHub, parallel execution, cross-browser testing, and advanced reporting.

Overall, this project demonstrates how Selenium WebDriver and Python can be combined with a structured testing framework to create a maintainable, reusable, and scalable web automation solution.

2. PROJECT OBJECTIVES

The primary objective of this project is to design and develop a structured Selenium Python Automation Framework capable of automating important functional scenarios of an E-Commerce web application. The project focuses on applying software testing principles and automation framework design techniques rather than developing isolated Selenium scripts.

The first objective is to automate web application functionality using Selenium WebDriver. Selenium provides the capability to interact with web browsers programmatically. Through Selenium, the framework can open the application, identify web elements, enter information, click buttons, navigate between pages, and verify application responses. This allows repetitive manual operations to be transformed into automated test scenarios.

The second objective is to use Python as the programming language for automation development. Python provides a simple and readable syntax and has extensive support for testing and automation libraries. Its integration with Selenium and PyTest makes it suitable for creating maintainable automation frameworks.

Another major objective is the implementation of the Page Object Model. POM is used to separate page-specific information from test-case logic. Each major application page is represented by a dedicated Python class. Locators and actions related to login, homepage operations, product search, and cart management are maintained separately. This improves code readability and reduces duplication.

The project also aims to implement PyTest as the test execution framework. PyTest provides fixtures, assertions, test discovery, markers, and reporting capabilities. Fixtures are used for common setup and teardown operations such as creating and closing the Selenium WebDriver. This prevents repetitive browser initialization code from being written in every test case.

Another objective is to implement data-driven testing. Test data is maintained separately in a CSV file rather than being directly embedded in test scripts. The framework reads this data through a reusable CSV reader utility. This allows the same test logic to be executed with different input values without modifying the source code.

The project also aims to implement configuration management. Application URL, browser selection, timeout values, and headless execution settings are maintained in a configuration file.

This approach makes the framework flexible and avoids hard-coding environment-specific values throughout the project.

An additional objective is to implement failure handling. When a test case fails, the framework automatically captures a screenshot of the browser state. This screenshot provides useful visual information for debugging and helps identify the state of the application at the time of failure.

The framework also incorporates logging to record execution-related information and HTML reporting through `pytest-html`. Finally, the project is intended to provide a professional-style automation framework that demonstrates Selenium automation, Python programming, PyTest testing, POM architecture, reusable utilities, data management, configuration management, error handling, logging, and reporting.

3. PROBLEM STATEMENT AND BUSINESS SCENARIO

Testing an E-Commerce web application involves repeatedly executing several user operations. A typical customer may need to open the website, log into an account, search for a product, select the product, add it to the shopping cart, verify the cart, and update the product quantity. When these operations are performed manually for every software release or build, the testing process becomes time-consuming and repetitive.

The problem addressed by this project is the development of an automated testing solution capable of performing these repetitive E-Commerce operations consistently. The framework should reduce manual effort while maintaining clear test organization and providing useful execution results.

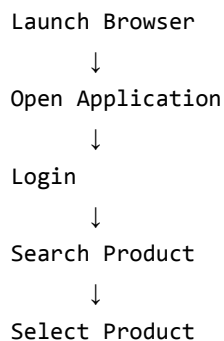
The selected business scenario is based on a customer purchasing a product from an E-Commerce website. The application used for the project is the TutorialsNinja Demo E-Commerce application. It provides an appropriate environment for demonstrating the required automation activities.

The first operation is launching the web browser and opening the application. The automation framework creates a Selenium WebDriver instance and navigates to the configured application URL. The next operation is user login. Login functionality is tested using both valid and invalid credentials.

After login-related testing, the framework performs product search functionality. The user enters the required product name into the application's search field. Selenium interacts with the search field and search button, after which the framework verifies that the expected product appears in the search results.

The next stage involves selecting the product and adding it to the shopping cart. The framework identifies the required product and performs the Add to Cart operation. Cart verification then checks product information and provides functionality for updating product quantity.

The overall business flow is represented below:





4. TECHNOLOGIES AND TOOLS USED

The Selenium Python Automation Framework has been developed using several software technologies and testing tools. Each technology performs a specific role within the framework and contributes to the overall automation solution.

Python is the primary programming language. Python was selected because of its readable syntax, extensive library support, and compatibility with Selenium and PyTest. The project environment uses Python 3.12.3. Python is used to create Page Object classes, test cases, utility modules, configuration readers, CSV readers, screenshot functionality, logging functionality, and framework fixtures.

Selenium WebDriver is the main web automation tool. It provides an interface for controlling web browsers programmatically and allows the framework to locate web elements, enter data, click buttons, navigate between pages, and retrieve information from the application. The project uses Selenium 4.35.0 with Google Chrome as the primary browser.

PyTest is the testing framework used to organize and execute automation test cases. It provides test discovery, assertions, fixtures, markers, and integration with reporting plugins. The project uses PyTest 8.4.2.

pytest-html version 4.1.1 is used to generate HTML execution reports. CSV is used for external test data, while an INI configuration file stores application URL, browser, timeout, and headless settings. Visual Studio Code can be used as the development environment for managing the Python project.

The main dependencies are:

Technology	Version	Purpose
Python	3.12.3	Programming language
Selenium	4.35.0	Web automation
PyTest	8.4.2	Test execution
pytest-html	4.1.1	HTML reporting
CSV	—	Test data
Google Chrome	Current	Browser under test

5. SYSTEM ARCHITECTURE

The system architecture of the project is designed to provide clear separation between test cases, page objects, utilities, configuration, and test data. The architecture follows a layered approach that improves maintainability and makes the framework easier to expand.

The test layer contains the actual automation scenarios. Test files such as `test_login.py` and `test_product_search.py` contain the business-level test logic and assertions.

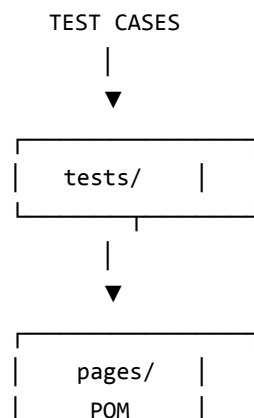
The Page Object layer contains classes representing application pages. The `LoginPage` class manages login operations, `HomePage` manages search functionality, `SearchPage` handles product-related operations, and `CartPage` handles cart-related functionality.

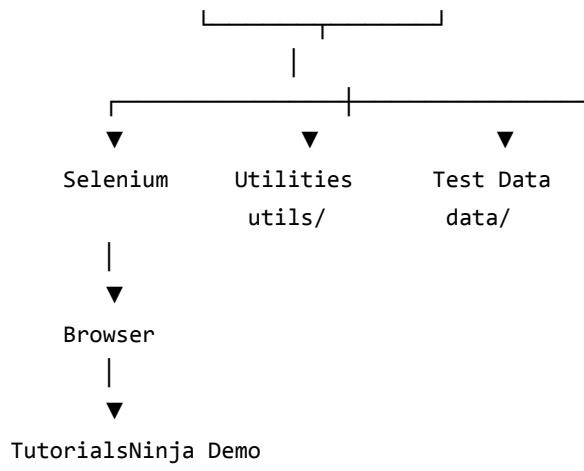
The utility layer provides reusable framework functionality. The `DriverFactory` creates the browser driver, `ConfigReader` reads configuration information, `CSVReader` reads test data, `Screenshot` manages failure screenshots, and `Logger` provides logging functionality.

The configuration layer stores environment-related settings. Instead of writing the application URL and browser configuration directly inside multiple files, these values are stored in `config.ini`. The test data layer stores external input values in a CSV file.

The execution layer consists of Selenium WebDriver and the Chrome browser. Selenium receives commands from the Page Object classes and performs the corresponding actions on the application. Reporting and debugging are supported through HTML reports, screenshots, and logs.

The architecture also provides a foundation for future CI/CD implementation. A Jenkins pipeline can execute the PyTest command, generate reports, and store execution results automatically. Therefore, the architecture provides a clear separation of responsibilities and forms the foundation of a scalable Selenium automation framework.





6. PROJECT STRUCTURE

The project follows a modular directory structure in which different types of files are organized according to their responsibilities. This organization improves readability, maintainability, and project management.

The config directory contains configuration information. The config.ini file stores the application URL, browser, timeout, and headless execution settings. The data directory contains the CSV test data file. Keeping data separately allows the framework to perform data-driven testing without changing the actual test implementation.

The pages directory contains the Page Object classes. login_page.py contains login-related locators and methods. home_page.py contains homepage and search-related actions. search_page.py handles product search results and product selection. cart_page.py manages cart-related actions.

The tests directory contains the actual test cases. test_login.py contains valid and invalid login scenarios, while test_product_search.py contains product search and shopping-cart scenarios.

The utils directory contains reusable components. driver_factory.py manages WebDriver creation. config_reader.py reads configuration values. csv_reader.py reads external test data. screenshot.py captures screenshots during failures, and logger.py manages logging.

The screenshots directory stores screenshots generated during failed test executions. The reports directory contains generated HTML reports. The logs directory stores execution logs.

The conftest.py file contains pytest fixtures and hooks used across test cases. pytest.ini provides pytest configuration and reporting settings. requirements.txt contains required package versions, README.md documents the project, and .gitignore prevents unnecessary generated files from being committed to version control.

This structure creates a clear separation between different components of the automation framework and makes the project easier for another developer or tester to understand.

```
Selenium_Python_Automation_Framework/  
|  
├── config/  
├── data/  
├── pages/  
├── tests/  
├── utils/  
└── screenshots/
```

- |— reports/
- |— logs/
- |— conftest.py
- |— pytest.ini
- |— requirements.txt
- |— README.md
- |— .gitignore

7. PAGE OBJECT MODEL IMPLEMENTATION

The Page Object Model (POM) is the central design pattern used in this project. POM organizes Selenium automation code by representing each important application page as a separate class.

Without POM, test scripts often contain locators, browser commands, page navigation, test data, and assertions together. As the number of tests increases, such scripts become difficult to maintain. If a locator changes, the same locator may need to be updated in several test files.

The LoginPage class contains elements and operations required for authentication. It includes methods such as `open_login_page()`, `enter_email()`, `enter_password()`, `click_login()`, `login()`, and `get_warning_message()`.

The HomePage class handles homepage functionality, particularly product searching. Its primary operation is `search_product()`, which enters the requested product into the search field and activates the search operation.

The SearchPage class manages product search results. It provides methods to retrieve the product name, select a product, add the product to the cart, and retrieve success information.

The CartPage class manages shopping-cart operations. It provides methods for opening the cart, retrieving product name and quantity, updating quantity, retrieving cart information, and removing products.

One major benefit of POM is readability. Instead of writing multiple Selenium commands, a test can call methods such as `home_page.search_product(product)` or `search_page.add_to_cart()`. This makes the test resemble the actual business process.

POM also improves maintainability. If a web element's locator changes, the relevant Page Object class can be updated without rewriting every test that uses that element. Explicit waits are implemented inside Page Objects to improve synchronization between the automation script and the application.

Overall, the Page Object Model provides a clean separation between application interaction and test logic and is therefore an important component of the project's maintainable automation architecture.

8. TEST CASE DESIGN AND AUTOMATION FLOW

Test cases are designed to validate important business functionalities of the E-Commerce application. The framework uses PyTest to execute the test cases and assertions to determine whether the actual application behaviour matches the expected result.

Test Case 1 — Valid Login: The test opens the login page, enters valid email and password values, clicks Login, and verifies successful login behaviour.

Test Case 2 — Invalid Login: The test enters invalid credentials, performs the Login operation, retrieves the warning message, and verifies that an appropriate warning response is displayed.

Test Case 3 — Product Search: The test obtains the product value from CSV test data, searches for the product, retrieves the displayed product name, and compares it with the expected value.

Test Case 4 — Add Product to Cart: The scenario searches for a product, selects it, and performs the Add to Cart operation.

Test Case 5 — Update Cart Quantity: The scenario searches for a product, selects it, adds it to the cart, opens the cart, changes the quantity, and verifies the resulting quantity.

The overall execution flow is: PyTest Fixture → Create Chrome Driver → Open Application → Execute Page Object Methods → Perform Application Actions → Assertions → Test Result → Screenshot if Failed → Close Browser.

The use of separate test cases allows individual functionalities to be executed independently. PyTest markers such as login and product can also be used to categorize tests. Assertions are essential because automation should not only perform actions but also verify application behaviour.

Test Case ID	Scenario	Expected Result
TC LOGIN 001	Valid Login	Successful login state
TC LOGIN 002	Invalid Login	Warning message displayed
TC PRODUCT 001	Product Search	Expected product displayed
TC PRODUCT 002	Add to Cart	Product added to cart
TC PRODUCT 003	Update Quantity	Quantity updated successfully

9. DATA-DRIVEN TESTING AND CONFIGURATION MANAGEMENT

Data-driven testing is an important component of the automation framework. Instead of hard-coding all test values inside test scripts, the framework stores test data separately in a CSV file.

The project's test data file is `data/test_data.csv`. An example is: `email,password,product` followed by `test@example.com,Test@123,MacBook`. The `CSVReader` utility reads this information and returns the required values to the test cases.

The primary advantage of data-driven testing is reusability. If the product name needs to be changed, the tester can modify the CSV file instead of changing the Python test code. Additional rows can be introduced for different scenarios.

Configuration management is another important part of the project. Configuration values are stored in `config/config.ini`. The file contains values such as `base_url`, `browser`, `timeout`, and `headless` execution settings.

The `ConfigReader` utility is responsible for reading these values. Separating configuration from code avoids hard-coded values throughout the framework and allows the same framework to be used with different environments by changing configuration values.

The `DriverFactory` uses the browser configuration to create the appropriate `WebDriver`. The framework also contains a `headless` option, allowing the browser to run without displaying the user interface when required.

The combination of test-data management and configuration management makes the framework more flexible. Test inputs are separated from test logic, while environment settings are separated from both.

This approach is particularly useful for larger automation projects where multiple environments, browsers, and test-data combinations may be required.

10. ERROR HANDLING, LOGGING AND SCREENSHOTS

A reliable automation framework should provide useful information when a test fails. A simple pass/fail result may not be sufficient to identify the reason for failure. Therefore, this project incorporates screenshot capture and logging mechanisms.

The framework uses a PyTest hook in `confest.py` to identify failed test cases. When a test fails during execution, the framework obtains the WebDriver instance and calls the screenshot utility. Screenshots are stored in the `screenshots` directory with the test name and timestamp.

The screenshot provides a visual representation of the browser state at the time of failure. For example, if a product test fails because a button cannot be located, the screenshot can help determine whether the expected page was loaded, whether a popup appeared, or whether the application displayed a different state.

The framework also contains a logging utility implemented using Python's logging module. Logs are stored in `logs/automation.log`. Logging can record information about important framework operations and execution events.

The project uses explicit waits to handle synchronization between Selenium and the web application. Web pages may take time to load elements, and immediate interaction can result in errors. The framework therefore uses `WebDriverWait` with expected conditions such as element visibility and clickability.

During development, some product-related automation scenarios encountered `TimeoutException` errors due to element-location and synchronization issues. These failures demonstrate why robust locators, appropriate waits, screenshots, and logs are important in Selenium automation.

The framework's error-handling approach can therefore be summarized as: Test Execution → Failure Detected → PyTest Hook → Capture Screenshot → Store Screenshot → Record Execution Information → Generate HTML Report.

These mechanisms make debugging more systematic. Instead of manually reproducing every failure, the tester can inspect the execution report, log information, and screenshot to understand the application state.

11. TEST EXECUTION AND RESULTS

The developed automation framework was executed using Python 3.12.3, PyTest 8.4.2, Selenium 4.35.0, and pytest-html 4.1.1. The project uses a Python virtual environment to isolate the required dependencies.

The virtual environment is activated using `.\venv\Scripts\Activate.ps1` and dependencies are installed using `pip install -r requirements.txt`. Individual test modules can then be executed using PyTest.

For login testing, `pytest tests/test_login.py` is used. The login module contains two test cases: `test_valid_login` and `test_invalid_login`. During project development, both login tests were successfully executed with the result: 2 passed in 11.44s.

This confirms that the browser setup, Selenium WebDriver, PyTest fixture, LoginPage implementation, assertions, and HTML reporting mechanism were successfully integrated for the login module.

The product test module contains `test_product_search`, `test_add_product_to_cart`, and `test_update_cart_quantity`. The product-search scenario successfully executed during development. The Add to Cart and Update Cart Quantity scenarios required refinement of page locators and synchronization because Selenium encountered timeout conditions while interacting with the application.

Such failures are an expected part of developing UI automation because web elements may have dynamic behaviour and the application structure may require careful locator selection. The Page Object Model makes such changes easier because page-specific implementation details are isolated from the test cases.

HTML reporting can be generated using `pytest --html=reports/report.html --self-contained-html`. The report is stored in the reports directory and provides an organized summary of test execution.

The testing process demonstrates that the framework is capable of launching the browser, loading the application, executing Page Object methods, performing assertions, recording test results, and generating reports.

Module	Result	Remarks
Valid Login	PASSED	Executed successfully
Invalid Login	PASSED	Executed successfully
Product Search	PASSED	Search scenario executed

Add Product to Cart	Under refinement	Locator/synchronization refinement
Update Cart Quantity	Under refinement	Locator/synchronization refinement

12. HTML REPORTING AND PROJECT OUTPUT

Reporting is an important part of any automation framework because test execution results need to be communicated clearly to developers, testers, and other stakeholders. This project uses the `pytest-html` plugin to generate HTML reports.

The HTML reporting mechanism is configured through `pytest.ini`. When the test suite is executed, PyTest records the results and generates an HTML document. The report can be generated using `pytest --html=reports/report.html --self-contained-html`.

The resulting file is stored in `reports/report.html`. The report provides a convenient way to review the execution outcome and can contain information about executed test cases, their status, environment details, and execution information.

The report is particularly useful when multiple test cases are executed because the tester does not need to rely only on terminal output. Instead, the HTML document provides a structured representation of the results.

The project also stores screenshots separately. When a test fails, the screenshot utility creates a timestamped image inside `screenshots/`. The screenshot can be reviewed alongside the HTML report to understand the application state at the time of failure.

The complete project output can therefore be organized as: Project → `reports/report.html`, `screenshots/failed_test_timestamp.png`, and `logs/automation.log`.

This organization creates a useful separation between execution results, visual evidence, and logging information. Reporting also makes the project more suitable for integration with continuous integration systems. In a future Jenkins implementation, Jenkins can execute the PyTest command and preserve the generated HTML report as a build artifact.

The reporting system therefore contributes to both manual review and future automation pipeline integration.

report.html

What's new in Microsoft Edge

C:/Users/user/Downloads/Selenium_Python_Automation_Framework/Selenium_Python_Automation_Framework/reports/report.html?sort=result

report.html

Report generated on 24-Sep-2026 at 13:33:42 by pytest.html v4.1.1

Environment

Python

3.13.7

Platform

Windows-10-10.0.19045-SP0

Packages

•

pytest: 8.4.2

•

pluggy: 1.6.0

Plugins

•

html: 4.1.1

•

metadata: 3.1.1

Summary

5 tests took 00:00:17.

(Un)check the boxes to filter the results.

☐ 0 Failed

☒ 5 Passed

☐ 0 Skipped

☐ 0 Expected failures

☐ 0 Unexpected passes

☐ 0 Errors

☐ 0 Retruns

Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_login.py::test_valid_login	00:00:06	
Passed	tests/test_login.py::test_invalid_login	00:00:02	
Passed	tests/test_product_search.py::test_product_search	00:00:01	
Passed	tests/test_product_search.py::test_add_product_to_cart	00:00:03	
Passed	tests/test_product_search.py::test_update_cart_quantity	00:00:05	

13. ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

The Selenium Python Automation Framework provides several advantages compared with basic Selenium scripts. The primary advantage is maintainability. The Page Object Model separates application-specific operations from test cases, making the framework easier to modify when the application changes.

Another advantage is reusability. Common operations such as WebDriver creation, configuration reading, CSV data reading, screenshot capture, and logging are implemented as reusable components. These utilities can be used by multiple test cases without duplicating code.

The framework also supports data-driven testing through CSV files. This makes it possible to change test inputs without modifying the automation logic. Configuration management provides similar flexibility by keeping environment settings separate from source code.

Failure screenshots and logging improve debugging. When an automation test fails, the tester can inspect the screenshot and log information rather than manually reproducing the failure immediately. HTML reporting provides a convenient mechanism for reviewing test results.

The current framework also has limitations. The present implementation primarily focuses on Chrome browser automation. Cross-browser execution with Firefox and Edge can be added in future versions. The current test data set is also limited and can be expanded to support multiple scenarios.

Another area for improvement is parallel execution. As the number of test cases increases, executing tests sequentially can require more time. PyTest can be extended with parallel execution capabilities to reduce overall execution time.

The framework can also be integrated with Jenkins for continuous integration and continuous testing. A Jenkins pipeline could automatically obtain the latest project version, create the testing environment, execute PyTest, and publish the generated report. GitHub integration can also be used for version control and collaborative development.

Additional E-Commerce scenarios can be implemented, including product removal, checkout, address management, order placement, and logout. API automation using Python Requests and advanced reporting tools such as Allure can also be introduced.

Future development can therefore combine Selenium UI automation, API automation, PyTest, GitHub, Jenkins CI/CD, and advanced reporting to create a more complete test automation framework.

14. CONCLUSION

The Selenium Python Automation Framework project demonstrates the development of a structured web automation solution for an E-Commerce application. The project combines Selenium WebDriver, Python, PyTest, Page Object Model, CSV-based test data, configuration management, explicit waits, screenshots, logging, and HTML reporting into a single automation framework.

The project was developed with the objective of moving beyond simple Selenium scripts and demonstrating the principles of framework-based automation. The Page Object Model provides a clear separation between page interaction and test logic. This improves readability and makes the framework easier to maintain and extend.

The automation framework covers important E-Commerce scenarios including valid login, invalid login, product search, product selection, adding products to the shopping cart, and updating product quantities. PyTest is used to organize and execute these scenarios, while fixtures provide common browser setup and teardown functionality.

The implementation of external test data through CSV files provides data-driven testing capabilities. Configuration management allows application and browser settings to be maintained separately from the source code. These features contribute to the flexibility and maintainability of the framework.

The project also demonstrates the importance of synchronization in UI automation. Explicit waits are used to ensure that web elements are ready before interaction. During development, product-related tests required locator and synchronization refinement, highlighting the practical challenges involved in Selenium automation.

Failure screenshots and logging provide valuable debugging support. HTML reports provide a structured view of test execution results. Together, these features make the framework more useful for real-world testing environments.

The successful execution of the login test module demonstrates that the core framework components have been integrated successfully. The project also provides a foundation for extending the automation solution to additional test scenarios.

In future development, the framework can be enhanced with cross-browser testing, parallel execution, API automation, Jenkins CI/CD integration, GitHub workflows, advanced reporting, and additional E-Commerce scenarios.

Overall, the project achieves its primary objective of demonstrating a structured and reusable Selenium Python automation framework for an E-Commerce web application.

15. REFERENCES

The following resources were used as technical references during the development and understanding of the automation framework.

1. Selenium WebDriver Documentation — <https://www.selenium.dev/documentation/>
2. Python Documentation — <https://docs.python.org/>
3. PyTest Documentation — <https://docs.pytest.org/>
4. pytest-html Documentation — <https://pytest-html.readthedocs.io/>
5. TutorialsNinja Demo E-Commerce Application — <https://tutorialsninja.com/demo/>

Selenium documentation was used for understanding WebDriver, browser automation, locators, waits, WebElement interactions, and Selenium's Python API. Python documentation was used for programming concepts, modules, file handling, configuration handling, CSV processing, exception handling, and logging.

PyTest documentation was used for understanding test discovery, fixtures, assertions, markers, hooks, and test execution. pytest-html documentation was used for understanding HTML test reporting and report generation.

The TutorialsNinja Demo E-Commerce website was used as the application under test. Selenium's Python WebDriver functionality was used for browser initialization, element identification, navigation, interaction, and automation of E-Commerce workflows.

The Page Object Model design approach was used to separate web-page locators and operations from test-case implementation. Python's CSV functionality was used to maintain and read external test data, while Python's built-in logging functionality was used to provide execution information and assist in debugging automation failures.

These references provided the technical foundation required to design and implement the Selenium Python Automation Framework described in this project report.

Reference	Resource
Selenium	https://www.selenium.dev/documentation/
Python	https://docs.python.org/
PyTest	https://docs.pytest.org/
pytest-html	https://pytest-html.readthedocs.io/
TutorialsNinja	https://tutorialsninja.com/demo/

APPENDIX — PROJECT CONFIGURATION

A. Requirements

```
selenium==4.35.0
pytest==8.4.2
pytest-html==4.1.1
```

B. Configuration

```
[DEFAULT]
base_url = https://tutorialsninja.com/demo/
browser = chrome
timeout = 10
headless = false
```

C. Execution Commands

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest
pytest tests/test_login.py
pytest tests/test_product_search.py
pytest --html=reports/report.html --self-contained-html
```

D. Framework Summary

The final framework separates test cases, Page Objects, utilities, configuration, data, screenshots, reports, and logs. This structure provides a maintainable foundation for future expansion into cross-browser automation, CI/CD execution, API testing, parallel testing, and advanced reporting.